SMART-CONTRACT SECURITY REVIEW — POC-VALIDATED, INDEPENDENTLY CRITIQUED

🔐 Security Review — PrivacyPortal & PrivacyPortalFactory

coti-io/coti-contracts @ 8a0c4928004dac7a6c8d50bde58a022b6912f963 · PrivacyPortal & PrivacyPortalFactory · 2026-09-05 · rev 3 (PoC-validated · fixes implemented, independently reviewed and test-validated)

Target: `coti-io/coti-contracts` @ commit `8a0c4928004dac7a6c8d50bde58a022b6912f963` **Method:** pashov/skills `solidity-auditor` v3 (`c577eb7`) — 12 parallel specialist agents on `opus`, orchestrator dedup and four-gate judging (rev 1) → executable PoCs against the unmodified contracts + independent critique (rev 2) → fixes implemented in a patched copy of the contracts, validated by test, and re-reviewed independently (rev 3).
Date: 2026-09-05 · **Revision:** v3

Scope

Repository / commit	`coti-io/coti-contracts` @ `8a0c4928004dac7a6c8d50bde58a022b6912f963`
Files reviewed	`contracts/pod/privacy/PrivacyPortal.sol` · `contracts/pod/privacy/PrivacyPortalFactory.sol` (with `PodERC20.sol` , `PodErc20Mintable*.sol` , `PrivacyPortalFeeLib.sol` , `PortalFeeOracle.sol` , `PodErc20CotiMother.sol` read for context)
Rev 1 confidence threshold	80 (agent convergence, not exploitability)

Rev 1 dedup / judging notes. 25 unique (Contract, function) tuples in the raw agent output. Rejected at gating: `PrivacyPortal.rescueNative` (admin-only fee redirect), `PrivacyPortalFactory._revokeRole` (cosmetic), `PrivacyPortalFeeLib.packFeeConfig` naming. Dropped as cosmetic/admin-adversarial: the `requestWithdrawWithPermit` event-field mismatch and the `setLimits` "maxWithdraw = 0" freeze (rev 2 notes that this last one is strictly worse than #12 and should not have been dropped).

How this revision was produced

1. **Exploit PoCs (15 tests, all passing).** Every finding is a Hardhat test that drives the **unmodified** coti-contracts code — `PrivacyPortal` , `PrivacyPortalFactory` , `PodErc20MintableInitializable` (→ `PodErc20Mintable` → `PodERC20`), `PrivacyPortalFeeLib` , `PortalFeeOracle` and, for #5, `PodErc20CotiMother` — through constructor-passthrough harnesses that add no logic (`test/portal-poc/contracts/PortalHarnesses.sol`). The only stand-in is the source-chain inbox (`MockInboxForPortal.sol`): it reproduces the real request-id packing and per-target nonce of `InboxBase` and delivers callbacks as `msg.sender == inbox` , so the real `onlyInboxPeer` / `onlyInboxReturnLeg` gates, the real `transferCallback` / `transferError` / `invalidatePendingRequest` / `killStaleRequest` paths and real EIP-712 permits execute. Files: `test/portal-poc/findings-A.ts` (#1–#7), `findings-B.ts` (#8–#14).

2. **Independent critique #1** (separate session, no access to rev 1 reasoning): re-ran the suite, re-read the code, judged precondition realism, documented-design status, existing mitigations and fix implementability. Its verdict vocabulary: `CONFIRMED`, `CONFIRMED-DOWNGRADED`, `DESIGN-CHOICE`, `DUPLICATE`, `NOT-REPRODUCED`.
3. **Fixes implemented and validated (rev 3).** One concrete fix per finding was applied to a copy of the three contracts (`test/portal-poc/contracts/patched/*Fixed.sol`, every change marked `// FIX #n`, review-driven changes marked `review change`). `test/portal-poc/fixes.ts` re-runs each exploit scenario against the patched stack (15 tests, including regressions for the reviewer's adversarial cases Y1–Y4): the exploit must fail and the legitimate path must still work.
4. **Independent critique #2** (rev 3, separate session): re-judged real-issue vs false-positive from the code, reviewed every fix for correctness, completeness and new risk, and wrote four adversarial tests against the patched code. It agreed with every real/false-positive verdict, shipped 8 fixes as-is, required changes on 4 (#1, #2, #4, #13) and rejected 1 (#11). All required changes were applied and re-tested; the rejected fix was withdrawn. Its per-finding verdict is quoted under *Independent fix review*.

```
export SOLC_NATIVE=/path/to/solc-static-linux   # 0.8.28+commit.7893614a
NODE_OPTIONS='--max-old-space-size=8192' npx hardhat --config hardhat.config.portal-poc.ts test \
  test/portal-poc/findings-A.ts test/portal-poc/findings-B.ts test/portal-poc/fixes.ts
# → 31 passing: 16 exploit PoCs on the original code, 15 fix validations on the patched code
```

Severity policy. Likelihood × impact, with the *who must act* precondition weighed explicitly. Rev 1 confidence numbers are kept for traceability only; the **Final severity** column supersedes them.

Summary

#	Finding	Real issue?	Final severity	Fix (patched, `// FIX #n`)	Fix validated
1	Withdrawal stuck after pToken transfer Success	**Yes**	Medium	WETH fallback on ETH-send failure + `retargetStuckWithdrawal` (owner → self only; admin while paused → anywhere)	✔ X1a, X1b
2	Blacklist not enforced on the release leg	Yes (blacklist half); pause half is intended	Low	recipient **and** payer blacklist check in `_releaseWithdrawal` (via `bindingFactory`); no route-around via re-target; pause deliberately not checked	✔ X2
3	Remount minter rotation bricks `adminRefundPendingDeposit`	Yes, recoverable	Low	owner-authorized `invalidatePendingRequest` + factory forwarder	✔ X3
4	Cross-factory remount skips every old-portal guard	**Yes**	Medium	authority read from the token (try/catch probes); previous portal must be paused **and** retired; `retirePortal` (re-entrant-safe); `setPTokenMinter` needs a paused portal	✔ X4, Y2/Y4
5	`createPortal` live before COTI registration	Documented design choice with a real window	Low	new clone starts paused (`pauseByFactory`)	✔ X5

#	Finding	Real issue?	Final severity	Fix (patched, `// FIX #n`)	Fix validated
6	Remount has no in-flight-escrow check	Duplicate of #3	Informational	covered by #3; test-mock gap noted	—
7	Operator freezes withdrawals via unbounded `fixedFee`	Yes	Low	admin-set `maxOperatorFixedFee` ceiling on operator setters	✔ X7
8	`refundFailedDeposit` rejects `Failed`	Yes, framing corrected	Low	admin refund of a terminal `Failed` mint no longer needs a portal pause	✔ X8
9	Deposit limits / fee on requested vs measured amount	Yes, narrow	Low	checks evaluated on `received`	✔ X9
10	`rescueERC20` drains collateral committed to pending withdrawals	Yes, recoverable	Low	`outstandingWithdrawalTotal` reserve	✔ X10
11	Fee floor collapses to `fixedFee` /0 on zero oracle rate	Documented fail-open	Informational	**no code change** (a strict-mode flag was proposed, then withdrawn after review: it would brick the exit path on an oracle outage); monitor `usedDynamicPricing`	—
12	`setLimits` allows an unpartitionable withdraw window	Admin misconfiguration	Informational	`maxWithdraw >= 2·minWithdraw − 1` rule	✔ X12
13	`wrap` has no caller-side fee bound	Yes	Low	`wrapWithMaxFee(..., maxPortalFee)` + admin cap `maxAutoWrapPortalFee` on the ERC-7984 `wrap` (deprecated for direct use)	✔ X13
14	Request-id reuse across inbox rotation	**Yes**	Medium	single-use request ids in `PodERC20` + escrow/burn overwrite guards (burn double-count sub-claim now proven on the original code)	✔ X14, X14b

No Critical or High. Three Medium (#1, #4, #14), all with implemented, independently reviewed and test-validated fixes.

Findings

1. Withdrawal whose pToken transfer succeeded but whose payout reverts is permanently stuck

`PrivacyPortal._releaseWithdrawal` · rev 1 confidence 90 · Medium

Description. Once `pToken.requests(transferRequestId).status == Success`, a payout that reverts (a recipient that cannot receive ETH on a native-wrapped portal, or an issuer-blocklisted recipient on an ERC-20 portal) makes `_releaseWithdrawal` revert forever. `cancelFailedWithdrawal` needs `Failed` / `SystemFailed`,

`killStaleRequest` needs `Pending`, and no function can re-target the withdrawal. The user's pTokens sit in portal custody on COTI outside `pendingBurnAmount` (never burnable) and the collateral is never released.

Is it a real issue? Yes. The stuck state is created in a single successful transaction: `PodERC20.transferCallback` writes `Success` first (PodERC20.sol:336) and then invokes the portal through a low-level call whose failure is swallowed (`address(to).call(callbackData)`, :367-371). Nothing in the harness manufactures this; the terminal-status wall is the real `_setRequestStatus` guard (:630-637). Precondition is unprivileged. The realistic trigger is external: a token issuer (USDC-style) blocklisting the recipient while the COTI leg is in flight. Recovery is admin-only and pays `rescueRecipient`, not the user. Both independent critiques upheld it at Medium.

Example scenario. 1. Alice holds 100 pUSDC. She calls `requestWithdrawWithPermit(recipient = Alice, 100)`. The portal stores the withdrawal as `TransferPending` and asks COTI to move 100 pUSDC from Alice to the portal. 2. While that message is in flight, the USDC issuer blocklists Alice's address. 3. COTI settles the move: Alice's 100 pUSDC are now in the portal's custody. The inbox delivers `transferCallback`; the pToken writes `Success`, calls `portal.onPTokenTransferred`, the USDC transfer to Alice reverts, the pToken emits `RequestCallbackFailed` and the transaction succeeds. 4. Alice retries `triggerWithdrawalRelease`: same revert. Bob, a keeper, tries `cancelFailedWithdrawal`: `WithdrawTransferNotFailed`. The admin tries `killPTokenStaleRequest`: `RequestNotPending`. `burnAccumulatedPTokens` cannot touch the 100 pUSDC because `pendingBurnAmount` was never incremented. 5. Alice has neither pTokens nor USDC. The only movement possible is the admin's `rescueERC20`, which sends the 100 USDC to `rescueRecipient`.

PoC evidence — `findings-A.ts` › F1a (native, non-payable recipient) and F1b (ERC-20, issuer blocklist applied after the request). One successful callback receipt contains both `RequestStatusUpdated(Success)` and `RequestCallbackFailed`; afterwards `triggerWithdrawalRelease` reverts `EthTransferFailed` / `IssuerBlocked`, `cancelFailedWithdrawal` reverts `WithdrawTransferNotFailed`, `killPTokenStaleRequest` reverts `RequestNotPending` after 2 days, `pendingBurnAmount == 0`, and the only withdrawal-id entry points in the ABI are `onPTokenTransferred`, `triggerWithdrawalRelease`, `cancelFailedWithdrawal`.

Recommended fix (implemented in `PrivacyPortalFixed.sol`). Two parts, because the two triggers differ.

(a) Never let a non-payable recipient strand a native withdrawal — deliver the wrapped asset instead:

```
  if (nativeWrappedUnderlying) {
      IWrappedNative(address(underlyingToken)).withdraw(withdrawal.amount);
      (bool ok,) = payable(withdrawal.recipient).call{value: withdrawal.amount}("");
      if (!ok) {
-         revert EthTransferFailed();
+         // FIX #1a: re-wrap and deliver WETH instead of stranding the withdrawal.
+         IWrappedNative(address(underlyingToken)).deposit{value: withdrawal.amount}();
+         underlyingToken.safeTransfer(withdrawal.recipient, withdrawal.amount);
      }
  }
```

(b) Let the withdrawal owner pull a provably stuck payout **back to themself**, or let a factory admin (portal paused) re-target it anywhere. Only the destination changes; user, amount, request id and the `Success` precondition are untouched; payer and new recipient must pass the blacklist:

```
function retargetStuckWithdrawal(bytes32 withdrawalId, address newRecipient) external
nonReentrant {
    if (newRecipient == address(0)) revert InvalidAddress();
    Withdrawal storage withdrawal = withdrawals[withdrawalId];
    if (withdrawal.user == address(0)) revert UnknownWithdrawal(withdrawalId);
    if (withdrawal.status != WithdrawalStatus.TransferPending) revert
WithdrawalNotPending(withdrawalId, withdrawal.status);
    IPrivacyPortalFactory ctrl = _controllerFactory();
    bool isOwner = msg.sender == withdrawal.user;
    bool isAdmin = ctrl.isAdmin(msg.sender) && paused();
    if (!isOwner && !isAdmin) revert NotWithdrawalOwner(withdrawalId, msg.sender);
    if (!isAdmin && newRecipient != withdrawal.user) revert RetargetOnlyToSelf(withdrawalId);
// review change (Y1)
    if (pToken.requests(withdrawal.transferRequestId).status != IPodERC20.RequestStatus.Success)
revert WithdrawalNotStuck(withdrawalId);
    if (blacklisted[withdrawal.user] || ctrl.blacklisted(withdrawal.user)) revert
AddressBlacklisted(withdrawal.user); // review change
    if (blacklisted[newRecipient] || ctrl.blacklisted(newRecipient)) revert
AddressBlacklisted(newRecipient);
    address old = withdrawal.recipient;
    withdrawal.recipient = newRecipient;
    emit WithdrawalRetargeted(withdrawalId, old, newRecipient);
}
```

Rev 1's "Option A" alone was only half a fix: re-wrapping addresses the native case but not an ERC-20 transfer that itself reverts. Note also that the release path now runs inside the miner's callback gas budget; if that budget is too small for the WETH fallback the hook still fails harmlessly and `triggerWithdrawalRelease` completes it.

Independent fix review. SHIP-WITH-CHANGES → applied. The reviewer's adversarial case Y1: the first version let the requester re-target *any* third-party payout, i.e. revoke a payment that might still be deliverable (and the window can be opened on purpose by under-funding the callback fee). The owner path is now self-only; arbitrary destinations need a factory admin with the portal paused. Fix (a) was judged correct as-is (CEI holds, return data discarded, both entry points `nonReentrant`).

Fix validation — `fixes.ts` › X1a: the rejecting recipient ends up holding WETH, the withdrawal is `Released`, `pendingBurnAmount == amount`. X1b: a not-yet-settled withdrawal cannot be re-targeted (`WithdrawalNotStuck`); a stranger cannot (`NotWithdrawalOwner`); the owner cannot redirect to a third party (`RetargetOnlyToSelf`) but can pull it back to themself and anyone releases it; when the *owner* is the blocked party, the admin re-targets while paused and the release completes.

---

2. Blacklist (and pause) are enforced only at request time

`PrivacyPortal._releaseWithdrawal` · rev 1 confidence 85 · **Low**

Description. `_releaseWithdrawal` — reachable permissionlessly via `triggerWithdrawalRelease` and via the pToken callback — checks neither the per-portal/factory blacklist nor `paused()`, so a withdrawal requested before a blacklisting or pause still pays out afterwards.

Is it a real issue? Half of it. The **pause half is intended**: the repo's own test `PrivacyPortalFactory.portalRemount.test.ts:383-435` requires in-flight withdrawals to settle on a paused, retired portal, and the migration model at `PrivacyPortalFactory.sol:600-601` depends on it; adding a pause check would turn every in-flight withdrawal at pause time into finding #1. The **blacklist half is real but narrow**: only withdrawals requested *before* the listing slip through; request-time checks (`PrivacyPortal.sol:519, 1031-1035`) hold for everything else, and the comment at :1028-1030 only promises recipient checking at request time. Impact is a compliance window equal to the in-flight set. No collateral loss, no unbacked supply.

Example scenario. 1. Bob requests a 100 USDC withdrawal to himself. His pTokens start moving on COTI. 2. Two minutes later the compliance team adds Bob to the factory blacklist (his new deposit and withdrawal

requests now revert `AddressBlacklisted`) and pauses the portal. 3. The COTI leg settles; the callback pays Bob 100 USDC from the paused portal. Had the hook failed for any reason, Carol (anyone) could still call `triggerWithdrawalRelease` and pay Bob.

PoC evidence — `findings-A.ts` › F2: two withdrawals to `user2` requested while clean; `user2` then blacklisted on factory and portal and both portal and factory paused. The callback path releases withdrawal #1 to the blacklisted `user2` while paused; withdrawal #2 is released by a stranger via `triggerWithdrawalRelease`. Both payouts happen with `paused() == true` and `blacklisted(user2) == true`.

Recommended fix (implemented). Enforce the compliance list at settlement, read through `bindingFactory` so a retired portal keeps settling; do **not** check pause:

```
   if (requestStatus != IPodERC20.RequestStatus.Success) {
       revert PTokenTransferNotSuccessful(withdrawal.transferRequestId, requestStatus);
   }
+ // FIX #2: compliance applies at settlement too; pause deliberately NOT checked (in-flight
settlement on a
+ //         paused portal is the documented migration model).
+ IPrivacyPortalFactory ctrl = _controllerFactory();
+ if (blacklisted[withdrawal.recipient] || ctrl.blacklisted(withdrawal.recipient)) revert
AddressBlacklisted(withdrawal.recipient);
+ if (blacklisted[withdrawal.user]      || ctrl.blacklisted(withdrawal.user))      revert
AddressBlacklisted(withdrawal.user); // review change
```

A listed party's withdrawal then sits in the same state as #1 by design (funds frozen under compliance control); de-listing, or the #1 admin re-target path for a legitimately mis-listed recipient, releases it. `retargetStuckWithdrawal` refuses a listed payer, so the control cannot be routed around.

Independent fix review. SHIP-WITH-CHANGES → applied: (i) the payer (`withdrawal.user`) is now checked too, matching rev 1's own Option A; (ii) the re-target path applies the blacklist to the payer, closing the route-around the reviewer demonstrated. The reviewer confirmed the important non-regression: in-flight withdrawals still settle on a paused, retired portal (X10).

Fix validation — `fixes.ts` › X2: a recipient listed after the request is not paid (hook fails, trigger reverts `AddressBlacklisted`, balance unchanged); a listed *payer* is not paid and cannot re-target to themself; after de-listing both withdrawals release, while the portal is paused.

---

3. Remount rotates the pToken minter away, bricking the retired portal's `adminRefundPendingDeposit`

`PrivacyPortal.adminRefundPendingDeposit` · rev 1 confidence 85 · **Low**

Description. `adminRefundPendingDeposit` calls `pToken.invalidatePendingRequest` (minter-gated) whenever the mint is `Pending`, but `createPortalWithExistingPToken` moves `minter` to the new portal while every escrow stays on the old one, so the refund reverts `OnlyMinter(oldPortal)`.

Is it a real issue? Yes, but recoverable, so Low. The revert is real (`PrivacyPortal.sol:632-634` → `PodERC20.sol:573-574` → `PodErc20Mintable.sol:47-51`; minter moved at `PrivacyPortalFactory.sol:674`). But the PoC itself demonstrates three shipped, admin-only recovery paths: kill after `requestKillMinAge` (1 day), `setPTokenMinter` swap-back, or `setPTokenRequestKillMinAge(0)` then kill — and `adminRefundPendingDeposit` was admin-only to begin with. Ops speed bump, not depositor loss. Rev 1's Fix B (`pendingEscrowCount()`) **cannot be implemented**: escrows are only ever written `Pending` (:420, :468) or `Refunded` (:597, :638); a mint success never terminalizes one, so such a counter would block every remount forever.

Example scenario. 1. Alice deposits 100 USDC; the mint stays `Pending` because the COTI callback never arrives. 2. The team upgrades the portal: pause A, `createPortalWithExistingPToken` → portal B becomes the minter, A is retired. 3. The admin tries `A.adminRefundPendingDeposit(aliceRequest)` : `OnlyMinter(A)`. Alice waits until the 1-day kill age elapses, or the admin temporarily swaps the minter back (which stops B from minting meanwhile).

PoC evidence — `findings-A.ts` › F3: refund reverts `OnlyMinter(0x…oldPortal)` ; `refundFailedDeposit` reverts `DepositMintNotFailed` ; the new portal reverts `DepositEscrowInvalid` ; all three recovery paths exercised from a snapshot.

Recommended fix (implemented). Give the token's Ownable owner (the factory) the authority to terminalize, behind an admin-gated forwarder, and let the refund proceed once the request is terminal:

```
  // PodERC20.invalidatePendingRequest
  function invalidatePendingRequest(bytes32 requestId) external {
-     _checkMinter();
+     if (msg.sender != owner()) {   // FIX #3: owner (factory) may also invalidate
+         _checkMinter();
+     }

  // PrivacyPortalFactory
+ function invalidatePTokenPendingRequest(address pToken_, bytes32 requestId) external
onlyRole(DEFAULT_ADMIN_ROLE) {
+     _requireFactoryOwnedPToken(pToken_);
+     IPodERC20(pToken_).invalidatePendingRequest(requestId);
+ }
```

`adminRefundPendingDeposit` keeps calling `invalidatePendingRequest` while the mint is `Pending` (so the direct path still works for a live portal); after a remount the admin first calls the forwarder, the request becomes `Failed` , and the refund skips that branch.

Independent fix review. SHIP. The reviewer noted this grants DEFAULT_ADMIN no new capability: `killStaleRequest` is already owner-only and `setRequestKillMinAge(0)` already removes the age gate in one transaction, so the forwarder is a convenience, not an escalation. Two nits applied: the `@dev` no longer claims `whenPaused` , and the pause requirement is expressed as "unless the request is terminal" (so an unreachable `None` status is not treated as terminal).

Fix validation — `fixes.ts` › X3: after the remount the direct refund still reverts `OnlyMinter` ; a stranger cannot use the forwarder; the admin's forwarder call sets `Failed` ; the retired portal's refund then succeeds immediately.

---

4. Cross-factory remount skips every old-portal safety guard

`PrivacyPortalFactory.createPortalWithExistingPToken` · rev 1 confidence 80 · **Medium**

Description. The `OldPortalNotPaused` , native-mode and `retireDepositsForUpgrade` checks are nested under `if (oldPortal != address(0))` where `oldPortal` is read from *this* factory's `portalForPToken` , so on the documented `transferPTokenOwnership` → remount-on-new-factory path they are all skipped while `setMinter(newPortal)` still runs.

Is it a real issue? Yes. Root cause confirmed at `PrivacyPortalFactory.sol:622, 651-666, 674` . Precondition is two admin actions, but the repo's own test `createPortalWithExistingPToken.test.ts:84-124` performs exactly this hand-off and remount on an unpaused portal, and `:411` documents "after handoff, remount on the new factory" as supported — a blessed path whose safety rail is silently absent. Result: one pToken supply over two disjoint collateral pools, the old portal reporting "open" while deposits revert deep inside the pToken, and the new factory's admin unable to pause the old portal.

Example scenario. 1. Team A runs factory F1 with portal A (USDC, 1 M USDC of collateral). Team A hands the pToken to Team B: `F1.transferPTokenOwnership(pUSDC, F2)`. 2. Team B calls `F2.createPortalWithExistingPToken(USDC, pUSDC)` without anyone pausing A. It succeeds: the minter is now portal B, A is untouched. 3. Alice deposits on A: reverts `OnlyMinter(A)` although A shows `paused() == false` and `isDepositEnabled == true`. 4. Bob withdraws on A: works, A pays from its 1 M USDC. Team B cannot pause A (`OnlyFactoryAdmin`). 5. Team B unpauses B (0 USDC). Carol withdraws 100 pUSDC on B: the COTI leg settles, the release reverts on balance — finding #1's shape.

PoC evidence — `findings-A.ts` › F4 with two real factories and different admins: minter on B, A unpaused/undetached, deposit on A → `OnlyMinter`, withdrawal on A `Released`, F2 admin `A.pause()` → `OnlyFactoryAdmin`, B unpaused with zero collateral → withdrawal accepted, release reverts `ERC20InsufficientBalance`.

Recommended fix (implemented). Read authority from the token, not from the local mapping; refuse to attach a pToken whose current minter is a live portal anywhere; give the old factory's admin a way to retire a paused portal without cloning; and close the `setPTokenMinter` back door:

```
  // createPortalWithExistingPToken, before cloning
+ if (oldPortal == address(0)) _requirePreviousMinterClosed(existingPToken,
nativeWrappedUnderlying, true);
  // same-factory path: do not retire a portal twice
- IPrivacyPortal(oldPortal).retireDepositsForUpgrade();
+ if (PrivacyPortal(payable(oldPortal)).factory() != address(0))
IPrivacyPortal(oldPortal).retireDepositsForUpgrade();

+ /// Probes the pToken's current minter with try/catch: a minter that is not a portal cannot
brick the token.
+ function _requirePreviousMinterClosed(address existingPToken, bool nativeWrappedUnderlying,
bool requireRetired) private view {
+     address prevMinter = PodErc20Mintable(payable(existingPToken)).minter();
+     if (prevMinter == address(0) || prevMinter.code.length == 0) return;
+     bool isPaused; try IPrivacyPortal(prevMinter).paused() returns (bool p) { isPaused = p; }
catch { return; }
+     address prevFactory; try PrivacyPortal(payable(prevMinter)).factory() returns (address f) {
prevFactory = f; } catch { return; }
+     bool prevNative; try IPrivacyPortal(prevMinter).nativeWrappedUnderlying() returns (bool n)
{ prevNative = n; } catch { return; }
+     if (!isPaused) revert OldPortalNotPaused(prevMinter);
+     if (requireRetired && prevFactory != address(0)) revert OldPortalNotRetired(prevMinter);
+     if (prevNative != nativeWrappedUnderlying) revert NativeWrapMismatch(prevMinter,
prevNative, nativeWrappedUnderlying);
+ }

+ /// Old factory's admin: detach a paused portal before handing the pToken to another factory.
+ function retirePortal(address portal) external onlyRole(DEFAULT_ADMIN_ROLE) {
+     if (PrivacyPortal(payable(portal)).factory() != address(this)) revert
PortalNotFromThisFactory(portal);
+     if (!IPrivacyPortal(portal).paused()) revert OldPortalNotPaused(portal);
+     IPrivacyPortal(portal).retireDepositsForUpgrade();
+ }

  // setPTokenMinter: the emergency rotation may not leave a live, unpaused portal advertising
deposits it cannot mint for
+ address current = PodErc20Mintable(payable(pToken_)).minter();
+ if (current != newMinter_ && current != address(0) && current.code.length != 0) {
+     try IPrivacyPortal(current).paused() returns (bool p) { if (!p) revert
OldPortalNotPaused(current); } catch {}
+ }
```

`PrivacyPortal.retireDepositsForUpgrade` is made idempotent (a detached portal accepts a repeat call from its binding factory as a no-op), so `retirePortal` is not a one-way door.

Independent fix review. SHIP-WITH-CHANGES → applied. The first version had three defects the reviewer proved with tests: (Y2) unguarded `paused()` / `factory()` / `nativeWrappedUnderlying()` probes on an arbitrary minter would revert forever for a non-portal minter, bricking the pToken on every factory; (Y4) `retirePortal` cleared `factory`, after which the same factory's own remount reverted `OnlyPortalFactory`; and `setPTokenMinter` still rotated the minter with no pause coupling, bypassing the guard entirely. All three are fixed above.

Fix validation — `fixes.ts` › X4: F2's remount reverts `OldPortalNotPaused(A)`; after A is paused it reverts `OldPortalNotRetired(A)`; a stranger cannot `retirePortal`; after F1's admin retires A the remount succeeds, the minter rotates, and deposits on A are refused. Y2/Y4 regression: `setPTokenMinter` away from a live portal reverts `OldPortalNotPaused` and works once paused; a non-portal minter is tolerated; a same-factory remount after `retirePortal` succeeds.

---

5. `createPortal` returns a live, deposit-enabled portal before the one-way COTI registration has landed

`PrivacyPortalFactory.createPortal` · rev 1 confidence 80 · **Low** (documented design choice with a real window)

Description. Unlike the remount path, `createPortal` never pauses the clone and `initialize` hardcodes `isDepositEnabled = true`, so deposits made before the fire-and-forget `registerToken` message executes on COTI hit `TokenNotRegistered` (a retryable revert that leaves the mint `Pending`) and lock collateral behind an admin-only, pause-gated refund.

Is it a real issue? It is documented behaviour with a real, un-atomic window. `PrivacyPortalFactory.sol:530-542` states the function returns as soon as the message is submitted, that mints will hit `TokenNotRegistered` and stay `Pending`, and instructs operators to set `isDepositEnabled = false` immediately. That guidance cannot be enforced atomically: between `createPortal` (DEPLOYER_ROLE, the lowest-trust role) and the admin's follow-up, any user can lock collateral. Minor doc inaccuracy: the NatSpec says "leave … false" but `initialize` hardcodes `true`.

Example scenario. 1. A deployer calls `createPortal(DAI, …)`. The transaction returns; the registration message is still in the inbox queue. 2. Alice, watching for new portals, deposits 10 000 DAI in the next block. The mint message reaches COTI before registration: `TokenNotRegistered`, retried, `Pending`. 3. Alice's `refundFailedDeposit` reverts `DepositMintNotFailed`. Her DAI is refundable only by an admin after pausing the whole portal (or the mint eventually succeeds once registration lands and is retried).

PoC evidence — `findings-A.ts` › F5: `paused() == false`, `isDepositEnabled == true` right after `createPortal`; the registration request is one-way with a zero error selector; a deposit is accepted and stays `Pending`; on the real `PodErc20CotiMother`, `mintPublic` for the not-yet-registered token reverts `TokenNotRegistered(0x5d0028cf)` and passes the gate after `registerToken`.

Recommended fix (implemented). Start closed; the admin unpauses after observing `TokenRegistered` on COTI:

```
  IPrivacyPortal(portal).initialize(underlying, pToken, decimals, nativeWrappedUnderlying,
address(this));
+ IPrivacyPortal(portal).pauseByFactory();   // FIX #5
```

`unpause()` still works because `factory != 0` (`PrivacyPortal.sol:272-277`). Also fix the NatSpec.

Independent fix review. SHIP.

Fix validation — `fixes.ts` › X5: the clone is paused after `createPortal`; deposits revert `DepositsPaused`; after the admin unpauses, deposits work.

---

6. Minter rotation in the remount path inspects none of the retiring portal's in-flight escrows

`PrivacyPortalFactory.createPortalWithExistingPToken` · rev 1 confidence 80 · **Informational (duplicate of #3)**

Same root cause, precondition, impact and fix as #3; it should have been merged at rev 1's dedup stage. The genuinely new content is a **test-coverage** observation: `MockPodErc20MintableForPortal` has no `invalidatePendingRequest` at all, and `MockPodERC20ForPortal.invalidatePendingRequest` has no minter gate, so the shipped suite cannot catch #3 (`findings-A.ts` › F6 demonstrates both). Recommendation: add the minter gate to the mocks and a remount-with-Pending-escrow test; the #3 fix covers the behaviour. Independent fix review: agrees it is a duplicate.

---

7. Operator role can freeze all withdrawals via an unbounded `fixedFee` floor

`PrivacyPortal.setWithdrawFee` / `PrivacyPortalFactory.setDefaultWithdrawFee` · rev 1 confidence 75 · Low

Description. `packFeeConfig` caps `percentageBps` at 10% but bounds `fixedFee` only by `uint96`, so an OPERATOR_ROLE holder — documented for "routine fee-parameter updates" — can set a floor no wallet can pay and block every withdrawal request, with no `Paused` event and no rescue enablement.

Is it a real issue? Yes. A privilege-boundary violation: OPERATOR_ROLE is deliberately lower-trust than admin (`PrivacyPortalFactory.sol:32`), `pause()` is admin-only (:433; `PrivacyPortal.sol:266`), yet the operator obtains an equivalent silent stop. No theft, no permanent loss, reversible in one transaction, already-pending withdrawals still release.

Example scenario. 1. Mallory obtains the operator key (a hot key used for routine fee tuning). 2. She calls `setDefaultWithdrawFee(2^96 − 1, 0, 2^96 − 1)`: one transaction, every portal without an override. 3. Alice's withdrawal request reverts `InsufficientPortalFee(79228162514264337593543950335, 0)`. Monitoring sees no `Paused` event; `paused()` is `false`. Service resumes only when someone lowers the fee.

PoC evidence — `findings-A.ts` › F7: operator cannot pause; `setWithdrawFee(2^96−1, 0, 2^96−1)` accepted; all withdrawals revert; `paused()` false; `rescueERC20` still `ExpectedPause`; factory-wide variant identical; lowering restores.

Recommended fix (implemented). An admin-set ceiling for operator-configured fixed fees, on both the portal overrides and the factory defaults; admins are exempt:

```
+ uint256 public maxOperatorFixedFee = 0.1 ether;                       // factory, admin-settable
+ function setMaxOperatorFixedFee(uint256 ceiling) external onlyRole(DEFAULT_ADMIN_ROLE) {
maxOperatorFixedFee = ceiling; }
+ function _checkOperatorFixedFee(uint256 fixedFee) private view {
+     if (fixedFee > maxOperatorFixedFee && !hasRole(DEFAULT_ADMIN_ROLE, msg.sender)) revert
FixedFeeAboveCeiling(maxOperatorFixedFee, fixedFee);
+ }
  // called first in setDefaultDepositFee / setDefaultWithdrawFee; the portal setters read the
ceiling via bindingFactory
```

Independent fix review. SHIP. Deployment note from the reviewer: the patched portal hard-calls `bindingFactory.maxOperatorFixedFee()`, so the portal and factory implementations must be upgraded together (a fixed portal on an old factory would brick both fee setters).

Fix validation — `fixes.ts` › X7: operator's huge `fixedFee` reverts `FixedFeeAboveCeiling` on both portal and factory; an in-ceiling operator fee is accepted; the admin may exceed it; withdrawals keep working.

---

8. `refundFailedDeposit` rejects mint status `Failed`, which the protocol's own ops tools write

`PrivacyPortal.refundFailedDeposit` · rev 1 confidence 75 · **Low**

Description. The permissionless refund accepts only `SystemFailed`, but `killPTokenStaleRequest` / `invalidatePendingRequest` terminalize a stuck mint as `Failed`, so after ops clears a stale mint the only refund path is a full-portal pause plus admin refund — unlike `cancelFailedWithdrawal`, which accepts both terminal states. `DepositEscrowStatus.Failed` is never written anywhere.

Is it a real issue? Yes, with rev 1's framing corrected. "Clearing a stale mint *kills* the permissionless refund" is wrong: a `Pending` mint was never permissionlessly refundable either (the PoCs show `refundFailedDeposit` reverting for `Pending`), and the exclusion of `Failed` is documented at `IPrivacyPortal.sol:155-159` ("App raise / Failed is not refundable"). The kill strictly improves the situation. The residual is real: an asymmetry with the withdrawal side and an operational cost — pausing the whole portal to refund one depositor even though a terminal `Failed` can never become `Success`.

Example scenario. 1. Alice's mint is stuck `Pending`. After a day the admin runs `killPTokenStaleRequest`: status `Failed`. 2. Bob, a keeper, calls `refundFailedDeposit`: `DepositMintNotFailed(id, Failed)`. 3. To return Alice's 100 USDC the admin must pause the entire portal, refund, and unpause — an outage for every other user for one refund.

PoC evidence — `findings-B.ts` › F8: after the kill, `refundFailedDeposit` reverts `DepositMintNotFailed(id, 3)`; the admin refund reverts `ExpectedPause` until paused; the `SystemFailed` path stays permissionless; `cancelFailedWithdrawal` accepts a killed transfer.

Recommended fix (implemented). The pause is a speed bump against a *late Success*; require it only while that is still possible:

```
- function adminRefundPendingDeposit(bytes32 requestId) external override onlyFactoryAdmin
nonReentrant whenPaused {
+ function adminRefundPendingDeposit(bytes32 requestId) external override onlyFactoryAdmin
nonReentrant {
      ...
-     if (mintStatus == IPodERC20.RequestStatus.Pending) {
-         pToken.invalidatePendingRequest(requestId);
-     }
+     if (mintStatus != IPodERC20.RequestStatus.Failed && mintStatus !=
IPodERC20.RequestStatus.SystemFailed) {
+         if (!paused()) revert ExpectedPause();          // FIX #8: only a not-yet-terminal mint
needs the pause
+         if (mintStatus == IPodERC20.RequestStatus.Pending)
pToken.invalidatePendingRequest(requestId);
+     }
```

Keeping the permissionless path `SystemFailed`-only is correct: an app-level `Failed` (raise) must stay admin-reviewed. Also write or remove the dead `DepositEscrowStatus.Failed` branch.

Independent fix review. SHIP.

Fix validation — `fixes.ts` › X8: an unpaused admin refund of a `Pending` mint still reverts `ExpectedPause`; after the kill the refund succeeds with the portal open.

---

9. Deposit limits and the fee are checked on the requested `amount`, but the position is minted from the measured `received`

`PrivacyPortal._deposit` / `depositNative` · rev 1 confidence 75 · **Low**

Description. `_checkDepositLimits(amount)` and the fee validation run before the balance-delta measurement while escrow and mint use `received`, so with a fee-on-transfer underlying a deposit that passes `minDepositAmount` can create a position below `minWithdrawAmount`, and the percentage fee is charged on the pre-fee notional.

Is it a real issue? Yes, narrow. Mechanism confirmed (`PrivacyPortal.sol:404, 406` before `:412-414`; escrow/mint use `received` at `:419-425`). Needs a fee-on-transfer underlying (explicitly supported, "PP-06") **and** an admin setting `minWithdraw ≈ minDeposit`; under the defaults (`1` , `:258-259`) it cannot occur. Rev 2's PoC second half (withdraw 100 of 190 and call the 90 residue stuck) was self-inflicted and is dropped; the first half stands.

Example scenario. 1. The portal wraps a 5%-tax token with `minDeposit = minWithdraw = 100`. 2. Alice deposits exactly 100; 95 arrive; 95 pTokens are minted. 3. Alice can never withdraw: 95 < `minWithdraw`. If deposits are later disabled or the portal retired, the position is locked for good. She also paid the percentage fee on 100, not 95.

PoC evidence — `findings-B.ts` › F9: escrow amount 95e6; withdrawals of 95 and 99 revert `WithdrawBelowMinimum`; fee floor 1.00 native for 100 vs 0.95 for 95; declaring the fee for the measured amount reverts `InsufficientPortalFee`.

Recommended fix (implemented). Evaluate limits and fee on what arrived:

```
- _checkDepositLimits(amount);
- _validateAndCollectPortalFee(portalFee, amount, true);
  ... safeTransferFrom / WETH.deposit; received = balance delta ...
+ _checkDepositLimits(received);                        // FIX #9
+ _validateAndCollectPortalFee(portalFee, received, true);
```

Independent fix review. SHIP.

Fix validation — `fixes.ts` › X9: a 100 deposit on the 5% token reverts `DepositBelowMinimum`; a 106 deposit nets 100.7 and is accepted; the fee floor on `received` is lower than on the requested amount, and one wei below it reverts.

---

10. `rescueERC20` has no reserve for withdrawals already in `TransferPending`

`PrivacyPortal.rescueERC20` · rev 1 confidence 75 · **Low**

Description. The portal keeps no aggregate of in-flight withdrawals and `rescueERC20` excludes only the pToken, so the documented pause → remount → rescue-full-balance migration removes collateral that pending withdrawals still need; when those transfers settle `Success` the release reverts on balance and the user ends with neither leg.

Is it a real issue? Yes, recoverable. No obligation total exists because `pendingBurnAmount` is incremented *inside* the release (`:866`). The repo's migration test rescues the full balance (`portalRemount.test.ts:147-156`) while another repo test (`:383-435`) relies on in-flight withdrawals settling against that same balance — the PoC is the intersection. Recovery is a plain ERC-20 transfer back to the retired portal, after which anyone can release, so Low. The missing on-chain obligation counter is the real defect.

Example scenario. 1. Alice requests a 100 USDC withdrawal (pending on COTI). 2. The team migrates: pause A, remount to B, `A.rescueERC20(USDC, everything)` to the rescue wallet, fund B. 3. Alice's transfer settles; the

release on A reverts (0 USDC). `cancelFailedWithdrawal` is refused (status `Success`). Alice's pTokens are in A's custody, uncounted. Someone must send 100 USDC back to A by hand before `triggerWithdrawalRelease` works.

PoC evidence — `findings-B.ts` › F10: full-balance rescue succeeds; the settling callback's hook fails; `triggerWithdrawalRelease` reverts `ERC20InsufficientBalance`; `cancelFailedWithdrawal` reverts `WithdrawTransferNotFailed`; `pendingBurnAmount == 0`; a manual transfer back makes the release succeed.

Recommended fix (implemented). Track committed collateral and floor the rescue on it:

```
+ uint256 public outstandingWithdrawalTotal;            // += at request, -= at release and at
cancel
  // rescueERC20
+ if (token == address(underlyingToken)) {
+     uint256 bal = IERC20(token).balanceOf(address(this));
+     uint256 free = bal > outstandingWithdrawalTotal ? bal - outstandingWithdrawalTotal : 0;
+     if (amount > free) revert RescueExceedsFreeCollateral(free, amount);
+ }
```

Independent fix review. SHIP.

Fix validation — `fixes.ts` › X10: full-balance rescue reverts `RescueExceedsFreeCollateral`; rescuing the free part succeeds; the pending withdrawal still releases and the counter returns to 0.

---

11. Fee floor silently collapses to `fixedFee` (legally 0) when the oracle returns a zero rate

`PrivacyPortal._portalFeeFloor` · rev 1 confidence 75 · **Informational (documented fail-open)**

Description. `resolvePortalFee` returns `(fixedFee, false)` on any zero rate and `_portalFeeFloor` discards the flag, so an unpriced underlying (`createPortal` sets no peg), a cleared peg, or a stale-to-zero adapter degrades a configured percentage fee to the flat fee, which may be 0, with no revert and no event.

Is it a real issue? Documented design choice. Fail-open is stated three times: `IPodPriceOracle.sol:9-10`, `PortalFeeOracle.sol:9`, `PrivacyPortalFeeLib.sol:78`. Reverting on a zero rate would brick deposits *and the exit path* factory-wide on any feed outage, which the leads section flags as a hazard. Impact is protocol revenue only; `usedDynamicPricing` and `tokenPriceUpdatedAt` exist for monitoring.

Example scenario. The team lists token X with a 1% fee but forgets to set its USD peg. Alice deposits 1 M X paying 0 fee. Nothing reverts, nothing is emitted; the treasury notices weeks later.

PoC evidence — `findings-B.ts` › F11: priced → `InsufficientPortalFee(1e18, 0)`; after `clearTokenPriceUSD` the same withdrawal succeeds at fee 0; a new portal for an unpriced token accepts a deposit at fee 0 with 1% configured.

Recommended fix: none in code. Rev 3 first proposed an opt-in `requireDynamicPricing` strict mode. The independent fix review **rejected** it and it was withdrawn: the fee floor is consulted on the *withdrawal* path too, so enabling the flag during an oracle outage would revert the exit path factory-wide (exactly the hazard that makes fail-open the right default), and the check sat after the `priceOracle == 0` early return, so it gave false assurance when no oracle was configured. Operational recommendation instead: alert on `estimate*Fees(...).usedDynamicPricing == false` for percentage-configured portals, and require a peg before `createPortal` in the deployment runbook. If a strict mode is ever wanted, scope it to deposits only and evaluate it before the early return.

Independent fix review. REJECT (fix withdrawn). Verdict on the finding unchanged: documented design choice, Informational.

---

12. `setLimits` accepts `maxWithdraw < 2·minWithdraw − 1`, leaving balances in the gap unredeemable

`PrivacyPortal.setLimits` · rev 1 confidence 75 · **Informational (admin misconfiguration)**

Description. Each min/max pair is validated only against itself, so with `maxWithdraw < 2·minWithdraw − 1` some balances cannot be split into any legal sequence of withdrawals.

Is it a real issue? Only as admin misconfiguration. Nothing unprivileged can trigger it; the stranded residue is bounded by `minWithdraw − 1` per holder (withdraw `maxWithdraw` first); balances below `minWithdraw` are already unredeemable with no max at all; pTokens are transferable, so dust can be pooled. Rev 1 dropped the strictly worse `maxWithdraw = 0` freeze as admin-adversarial while promoting this — inconsistent gating.

Example scenario. The admin sets `(minWithdraw, maxWithdraw) = (100, 150)`. Alice holds 170: 170 exceeds the max, 150 then 20 fails the min, 100 then 70 fails the min. With deposits disabled, 20 stays locked unless she transfers pTokens to Bob to reach a splittable amount.

PoC evidence — `findings-B.ts` › F12.

Recommended fix (implemented). Every balance ≥ `minWithdraw` is splittable iff `maxWithdraw ≥ 2·minWithdraw − 1`:

```
+ if (maxWithdraw != 0 && minWithdraw > 1 && maxWithdraw < 2 * minWithdraw - 1) revert
InvalidLimitConfiguration();
```

Independent fix review. SHIP.

Fix validation — `fixes.ts` › X12: `(100, 150)` and `(100, 2·100 − 2 units)` rejected; `(100, 2·100 − 1 units)` and `(0, 0)` accepted.

---

13. `wrap` charges an execution-time fee with no caller-supplied bound

`PrivacyPortal.wrap` · rev 1 confidence 70 · **Low**

Description. `wrap` derives `portalFee` from the live floor instead of validating a caller-declared value, so a fee or oracle move between quote and inclusion is silently absorbed as protocol fee out of the forwarded mint budget, where `deposit` would have reverted `InsufficientPortalFee`.

Is it a real issue? Yes, bounded. Confirmed at `PrivacyPortal.sol:498` vs `:888-898`. Trigger is an operator fee change (the same semi-trusted role as #7) or an ordinary oracle move, which needs no privilege. Loss is capped by `maxFee`; if the squeeze pushes the mint budget under the inbox minimum the transaction reverts in production (`FeeManager.sol:164-170`; the mock inbox does not model minimums). `wrap` is the ERC-7984 integrator path; `deposit` remains the bounded primary path.

Example scenario. Alice's integration quotes 1.0 native and sends 1.5 (0.5 for the async mint). An operator raises the fee to 1.4% in the same block. `wrap` charges 1.4, leaves 0.1 for the mint leg; Alice overpays 0.4 with no revert and no way to have set a limit.

PoC evidence — `findings-B.ts` › F13: `deposit` with the stale quote reverts `InsufficientPortalFee`; `wrap` with the same value succeeds with `OperationFeesPaid.portalFee == 1.4e18` and `podFee` reduced by 0.4e18.

Recommended fix (implemented). A bounded variant for integrators that can pass a bound, plus an admin cap on the ERC-7984-shaped `wrap` (which keeps its signature and is deprecated for direct use):

```
function wrapWithMaxFee(address to, uint256 amount, uint256 mintCallbackFee, uint256
maxPortalFee) external payable nonReentrant returns (bytes32) {
    (uint256 portalFloor,) = _portalFeeFloor(amount, true);
    if (portalFloor > maxPortalFee) revert ExcessivePortalFee(maxPortalFee, portalFloor);
    return _deposit(to, amount, portalFloor, mintCallbackFee);
}

uint256 public maxAutoWrapPortalFee;                        // admin-set; 0 = no cap (current
behaviour)
function wrap(address to, uint256 amount, uint256 mintCallbackFee) external payable nonReentrant
returns (bytes32) {
    (uint256 portalFloor,) = _portalFeeFloor(amount, true);
    if (maxAutoWrapPortalFee != 0 && portalFloor > maxAutoWrapPortalFee) revert
ExcessivePortalFee(maxAutoWrapPortalFee, portalFloor);
    return _deposit(to, amount, portalFloor, mintCallbackFee);
}
```

Independent fix review. SHIP-WITH-CHANGES → applied. `wrapWithMaxFee` alone left the finding open because `wrap` is the entry point existing ERC-7984 integrations call; the reviewer asked for a deprecation note and an admin-settable cap that `wrap` enforces. Both added.

Fix validation — `fixes.ts` › X13: with a 1.0 bound the 1.4 floor reverts `ExcessivePortalFee`; with a 1.4 bound the wrap succeeds and charges 1.4; plain `wrap` still charges the live 1.4 floor by default and reverts once the admin sets `maxAutoWrapPortalFee = 1.0`.

---

14. Escrow, burn and withdrawal state are keyed on inbox request ids that are unique only per inbox instance

`PrivacyPortal._deposit` / `PodERC20._setRequestStatus` · rev 1 confidence 70 · **Medium**

Description. `depositEscrows[requestId]` and `burnInFlight[burnRequestId]` are written unconditionally and `PodERC20._setRequestStatus` does not guard writes of `Pending`, so if the admin re-points the pToken at a fresh inbox whose per-target nonce restarts, colliding ids overwrite live escrows, rewind terminal statuses, and can resurrect a stranded withdrawal's `transferRequestId` to `Success`, letting `triggerWithdrawalRelease` pay out collateral for a pToken transfer that never happened.

Is it a real issue? Yes. Every step is anchored in production code: `InboxBase._packRequestId` mixes only (source chain, target chain, nonce) with no inbox component (`InboxBase.sol:650-662`) and a fresh deployment restarts the per-target nonce at 1 (`:507`); escrows are assigned unconditionally (`PrivacyPortal.sol:420, :468`); `_setRequestStatus` guards only terminal writes (`PodERC20.sol:628-643`). Precondition narrows likelihood: a DEFAULT_ADMIN must `configurePToken` a live pToken to a **newly deployed** inbox (a proxy upgrade keeps address and nonce and is harmless). Where it fires the impact is severe and neither self-healing nor admin-recoverable. The `burnInFlightTotal` double-count sub-claim, left unverified in rev 2, is now proven by `findings-B.ts` › F14b: a reissued burn id overwrites `burnInFlight[id]` while `burnInFlightTotal` is incremented twice, one reservation is orphaned forever (`UnknownBatchBurn` after finalization) and later batch burns hit `PendingBurnTooLow`.

Example scenario. 1. Alice deposits 100 USDC (request nonce 2, `Success`). Later she requests a 50 USDC withdrawal (nonce 3); the COTI leg is delayed and stays `Pending`. 2. The team migrates to a freshly deployed inbox and calls `configurePToken(pUSDC, newInbox, mother)`. The new inbox starts at nonce 1. 3. Bob deposits twice. His second deposit takes nonce 2: it overwrites Alice's escrow record (Alice's refund claim now belongs to Bob) and rewinds her mint from `Success` to `Pending`. His third deposit takes nonce 3, Alice's withdrawal id. 4. Bob's mint settles `Success`. That flips Alice's stranded withdrawal to `Success`; Carol calls `triggerWithdrawalRelease` and the portal pays Alice 50 USDC while her 50 pUSDC never moved on COTI. The portal is now under-collateralized and `pendingBurnAmount` claims pTokens it never received.

PoC evidence — `findings-B.ts` › F14 with the real id scheme: escrow overwritten, `Success → Pending` rewind, stranded withdrawal paid, `pendingBurnAmount` inflated. F14b: burn-id collision leaves `burnInFlightTotal` permanently inflated by the first reservation.

Recommended fix (implemented). Make request ids single-use at the pToken, and refuse to overwrite live records at the portal:

```
  // PodERC20._setRequestStatus
+ if (status == IPodERC20.RequestStatus.Pending && current != IPodERC20.RequestStatus.None) {
+     revert RequestIdAlreadyUsed(requestId, current);     // FIX #14
+ }
  // PrivacyPortal._deposit / depositNative
+ if (depositEscrows[requestId].status != DepositEscrowStatus.None) revert
RequestIdAlreadyUsed(requestId);
  // PrivacyPortal.burnAccumulatedPTokens
+ if (burnInFlight[burnRequestId] != 0) revert RequestIdAlreadyUsed(burnRequestId);
```

Consequence (documented trade-off): a from-scratch inbox whose nonces overlap the pToken's history cannot be used for that pToken — every colliding send reverts until the new inbox's per-target nonce climbs past the highest id the pToken has seen — which is a temporary, loud DoS instead of silent corruption. The inbox itself should also mix its own address into request ids. The portal-side guards are unreachable through mint/burn because the pToken reverts first; they are belt-and-braces.

Independent fix review. SHIP. The reviewer called `_setRequestStatus` the right choke point (every send funnels through it, fail-closed, no extra SLOAD) and asked for the burn sub-claim to be proven and the DoS trade-off to be documented; both done.

Fix validation — `fixes.ts` › X14: after rotation, the first fresh nonce works; the colliding deposit reverts `RequestIdAlreadyUsed` before any portal state is written; Alice's escrow and `Success` status are intact; her stranded withdrawal stays `Pending` and cannot be released. X14b: a colliding burn id is refused and `burnInFlightTotal` is not inflated.

Independent fix review — summary

#	Real?	Severity	Fix verdict (first version)	Change applied
1	REAL	Medium	SHIP-WITH-CHANGES	owner re-target restricted to self (Y1)
2	REAL (blacklist) / DESIGN (pause)	Low	SHIP-WITH-CHANGES	payer checked at release; re-target refuses a listed payer
3	REAL	Low	SHIP	NatSpec + "unless terminal" pause condition
4	REAL	Medium	SHIP-WITH-CHANGES	try/catch probes (Y2), idempotent retire (Y4), `setPTokenMinter` guard
5	DESIGN-CHOICE	Low	SHIP	—
6	DUPLICATE of #3	Informational	—	—
7	REAL	Low	SHIP	— (deploy portal + factory together)
8	REAL	Low	SHIP	—
9	REAL	Low	SHIP	—

#	Real?	Severity	Fix verdict (first version)	Change applied
10	REAL	Low	SHIP	—
11	DESIGN-CHOICE	Informational	**REJECT**	fix withdrawn (Y3)
12	REAL (admin misconfiguration)	Informational	SHIP	—
13	REAL	Low	SHIP-WITH-CHANGES	`wrap` deprecated + admin cap
14	REAL	Medium	SHIP	burn sub-claim proven (F14b / X14b); DoS trade-off documented

The reviewer agreed with every real / false-positive verdict of critique #1. Its four adversarial tests (Y1–Y4) against the first patched version all passed, i.e. all four defects were real; each is now covered by a regression test in `fixes.ts` and the corresponding change is marked `review change` in the patched sources. A remaining deployment note: `outstandingWithdrawalTotal` and `maxAutoWrapPortalFee` were inserted mid-contract in the patched copy; for an upgrade of live clones, append new storage at the end.

Corrections carried from rev 1 / rev 2

1. **#8's framing** ("kill destroys the permissionless refund") was wrong — a `Pending` mint was never permissionlessly refundable; the kill enables the admin refund.
2. **#3's rev 1 Fix B** (`pendingEscrowCount`) is unimplementable — escrows never terminalize on mint success.
3. **#1's rev 1 Fix A** covered only the native case; rev 3 pairs it with an owner re-target.
4. **#2** treated the intended pause bypass as a defect; rev 3 fixes only the blacklist half.
5. **#6** is a self-declared duplicate of #3; **#12** was promoted while a strictly worse `setLimits` freeze was dropped.
6. **#9's** second PoC half (withdraw 100 of 190) was self-inflicted and is no longer cited.

Additional observations (not scored)

- **`feeRecipient` is `immutable` with no setter** (`PrivacyPortalFactory.sol:52, 205`) while `withdrawPortalFees` hard-requires the send to succeed (`PrivacyPortal.sol:742-743`); if that address ever rejects ETH, every portal's fees are stranded factory-wide.
- **`setLimits` accepts `maxWithdraw == 0` / `maxDeposit == 0`** as a silent total freeze with no `Paused` event (#7's shape, one privilege level up).
- **No on-chain record of pTokens actually held on COTI**: `pendingBurnAmount` is incremented inside `_releaseWithdrawal` (`:866`), the shared root cause of #1's unburnable custody and #10's invisible obligation.
- **PoC fidelity caveat**: the mock inbox enforces no fee minimums (production `FeeManager` reverts `TotalFeeTooLow` / `CallbackFeeTooLow`); no finding's mechanism depends on this. Callback deliveries in the harness carry an explicit 5M gas budget because `eth_estimateGas` cannot see inside the swallowed hook call.
- **The 1-day kill delay** that #3 leans on is a one-transaction admin knob (`setPTokenRequestKillMinAge`).

Leads

Vulnerability trails with concrete code smells where the full exploit path could not be completed in one analysis pass. Not scored.

- **Permissionless release entry inherits the release-leg gap** — `triggerWithdrawalRelease` — the surface through which #2 fires and through which a resurrected `transferRequestId` (#14) would be cashed.
- `burnInFlight` **overwrite double-counts** `burnInFlightTotal` — `burnAccumulatedPTokens` — assignment beside `+=` with no occupancy check; on an id collision the total inflates permanently (guarded by the #14 fix).
- **Forgeable** `OnlyPToken` **guard** — `onPTokenTransferred` — `transferAndCall` lets any holder make the pToken issue arbitrary calldata; harmless today because the handler re-validates everything.
- **Unguarded oracle call on every user path, including exit** — `_portalFeeFloor` — a reverting or gas-bombing adapter bricks deposits and new withdrawals factory-wide.
- **Force-** `Failed` **transfer may close a withdrawal whose COTI leg settled** — `cancelFailedWithdrawal` after `killStaleRequest`.
- `pendingBurnAmount` **can be permanently over-stated** — `finalizeBatchBurn` only decrements on `Success`.
- **Outbound legs are unmeasured; refunds don't unwrap** — `_releaseWithdrawal` / `refundFailedDeposit`.
- **18-decimal assumption for the wrapped native** — `depositNative`.
- **Per-portal risk controls reset on remount** — `initialize`.
- **One-shot, unacknowledged mother registration; underlying slot can be squatted** — `createPortal`.
- **Rotating the COTI peer on a live pToken strands in-flight requests** — `configurePToken`.
- `setPTokenMinter` **leaves the mapped portal advertising itself live** — `setPTokenMinter`.
- **Constructor skips the code-existence checks the setters enforce** — `PrivacyPortalFactory.constructor`.

Appendix — files

Path	Role
`test/portal-poc/findings-A.ts`, `findings-B.ts`	exploit PoCs on the unmodified contracts (16 tests)
`test/portal-poc/fixes.ts`	fix validations on the patched contracts (15 tests, incl. Y1–Y4 regressions)
`test/portal-poc/helpers.ts`	fixture (real factory → real clones), EIP-712 permits, callback delivery
`test/portal-poc/contracts/MockInboxForPortal.sol`	inbox stand-in with the real request-id / nonce scheme
`test/portal-poc/contracts/PortalHarnesses.sol`	constructor-passthrough harnesses of the real contracts
`test/portal-poc/contracts/patched/*Fixed.sol`	patched copies; every change marked `// FIX #n` / `review change`
`audit/privacy-portal-critique-rev2.md`, `audit/privacy-portal-fix-review-rev3.md`	the two independent critiques, verbatim
`audit/diagrams/`	F1 call flow, refund state matrix, system architecture (independently verified)

⚠️ This review was performed by an AI assistant. AI analysis can never verify the complete absence of vulnerabilities and no guarantee of security is given. Team security reviews, bug bounty programs, and on-chain monitoring are strongly recommended. For a consultation regarding your projects' security, visit https://www.pashov.com